Quadrotor Control and Evaluation Simulation on NVIDIA Jetson

Ayberk Aygün* and Berkay Taşkın†
*Computer Engineering Department, Hacettepe University, Ankara, Türkiye
†Graduate School of Science and Engineering, Hacettepe University, Ankara, Türkiye
E-mails: *ayberk.aygun07@gmail.com, †berkaytaskinw@gmail.com

***Abstract*—Unmanned aerial vehicles rely heavily on control algorithms to maintain their stability and execute their missions, and these algorithms must run under hard real-time constraints on resource-limited on-board computers. This project studies that problem on a heterogeneous CPU–GPU embedded platform of the NVIDIA Jetson class, and is written from the CPU (host) side of the system. The host runs the nominal real-time loop — a smooth reference generator, a cascaded PID controller, a motor mixer, and a forward-Euler quadrotor simulation at a 5 ms step — and also acts as the coordinator that offloads an expensive PID gain search to the GPU and consumes its result. The offloaded accelerator is a re-engineered GPU Accelerated Trajectory Evaluation (GATE) pipeline in which each thread evaluates one candidate gain set sampled around an initial controller; a scoring stage ranks candidates by position RMSE, settling time, and overshoot, and the host selects the best set and updates the controller. We treat the partitioning between a real-time control loop and an asynchronous evaluation workload, the host-side orchestration cost, and the footprint of the offloaded kernels as the central embedded-design concerns. Starting from a deliberately unstable controller, the host recovers a stable, well-tracking gain set in three sub-second search stages, while the offloaded kernels stay within a compact per-thread budget (664 B and 604 B, both under 2 KB) suitable for a Jetson-class device.**

***Index Terms*—Quadrotor, PID, Controller Evaluation, Monte Carlo, Simulation.**

I. Introduction

Unmanned aerial vehicles (UAVs) have become increasingly prevalent across a wide range of applications including surveillance, delivery, search and rescue, and agricultural monitoring. The reliable operation of these vehicles depends heavily on the performance of their control algorithms, which must maintain stability and track desired trajectories under real-world conditions. In practice, drone controllers face numerous challenges such as sensor noise, external disturbances like wind gusts, model uncertainties in physical parameters, and variations in initial flight conditions. Designing a single controller that performs robustly across all these uncertainty sources remains a significant challenge in the field of UAV control.

A widely adopted approach for evaluating controller robustness is Monte Carlo simulation, where the controller is tested repeatedly under randomized scenarios by injecting perturbations drawn from known probability distributions and observing the resulting system behavior [1]. While this method provides statistically meaningful robustness metrics, it requires running a large number of independent trajectory simulations. When these simulations are executed sequentially on a CPU, the computational cost becomes prohibitive, limiting both the number of test scenarios that can be analyzed and the speed at which evaluation results become available. This limitation is particularly critical on embedded platforms where computational resources are already constrained.

Graphics processing units (GPUs) have been demonstrated as an effective platform for accelerating Monte Carlo simulations through parallelization. In the context of controller evaluation, the GPU Accelerated Trajectory Evaluation (GATE) framework [2] provides a C++/CUDA-based tool that distributes independent trajectory rollouts across GPU threads. GATE separates the simulation into three modular components — dynamics, guidance, and perturbations — each managing its own GPU memory through a managed base class. The framework employs template-based static polymorphism (CRTP) to avoid virtual function table overhead on the GPU, and provides a plug-and-play architecture where different controllers and dynamic models can be integrated without modifying the GPU backend.

Separately, prior work on real-time quadrotor simulation has addressed the challenge of running nonlinear quadrotor dynamics within tight execution time budgets by proposing computational complexity-based approaches [3]. These methods provide a foundation for implementing physically accurate simulations on platforms where timing constraints must be respected, which is directly relevant to embedded deployment scenarios.

In this project, we combine these two lines of work by adapting GATE's GPU-parallel Monte Carlo architecture [2] for use with a cascaded PID controller on an NVIDIA Jetson embedded platform. The Jetson board provides both multi-core ARM CPUs and an integrated CUDA-capable GPU within a single board, making it a suitable platform for heterogeneous workload partitioning. The quadrotor is modeled as a nonlinear system whose state comprises position, velocity, Euler attitude angles, and body-frame angular rates. A cascaded PID controller is designed following the standard approach used in real-world flight controllers: the outer loop converts position error into desired attitude references, and the inner loop tracks these references by commanding motor torques through a motor mixer. Controller gains are tuned offline through closed-loop pole placement analysis to ensure numerical stability with the forward Euler integrator at a 5 ms step size.

The system partitions the workload between CPU and GPU

following GATE's design philosophy [2], but in this report the CPU/host side is the protagonist. The CPU executes the nominal real-time control loop — reference generation, the cascaded PID controller, the motor mixer, and the forward-Euler dynamics — and simultaneously acts as the coordinator: it samples candidate PID gain sets, offloads their evaluation to the GPU, selects the best result, updates the controller, and drives the on-host visualization. The GPU is used as an offloaded coprocessor that evaluates many candidate gain sets in parallel and returns compact results. This closes the loop between gain search and controller tuning on the embedded platform. Unlike [1], which applies Monte Carlo methods within an MPC framework, our approach focuses on evaluating and adapting a cascaded PID controller on embedded hardware.

The main contributions of this work, seen from the embedded-systems side, are: a heterogeneous design in which the ARM CPU carries the hard real-time control and simulation loop and simultaneously orchestrates an offloaded GPU gain-search without giving up its real-time deadline; a host-side workflow (reference generation, candidate sampling, kernel launch and synchronization, best-gain selection, controller update, and visualization) that consumes the accelerator purely as an advisory coprocessor; and a demonstration that the offloaded kernels are compact enough to fit a Jetson-class device, verified at build time.

This report is written for the embedded-systems design course and concentrates on the CPU/host side of the system: the real-time control loop, the dynamics integration, the host-to-device orchestration, and the system-level concerns of timing, resource use, and reliability on a unified-memory SoC. The GPU accelerator is described where needed — it is an essential part of the architecture — but its internal kernel optimization is the subject of the companion parallel-computing treatment of the same project, whereas here it is treated mainly as the coprocessor that the CPU drives and whose results the CPU integrates. Compared with the original proposal, the implemented system also diverges in one respect: instead of injecting environmental perturbations (mass, inertia, control noise) for robustness evaluation, the current implementation disables those perturbations and uses the offloaded search to evaluate many candidate PID gain sets around an initial controller, so that the effect of gain selection can be isolated. The perturbation path remains in the architecture and is left as future work.

II. Related Work

Our project has several key characteristics. It is a quadrotor dynamic model running on the Jetson CPU producing a 6-DoF Quadrotor simulation. It has six decoupled PID controllers that govern the quadrotor orientation. GPU-accelerated Monte Carlo simulation on the Jetson GPU to determine the best set of PID gains in parallel, and finally an embedded architecture where both simulation and optimization reside on a single Jetson board, enabling potential on board real-time tuning. We have found some work relating to some of these aspects. The

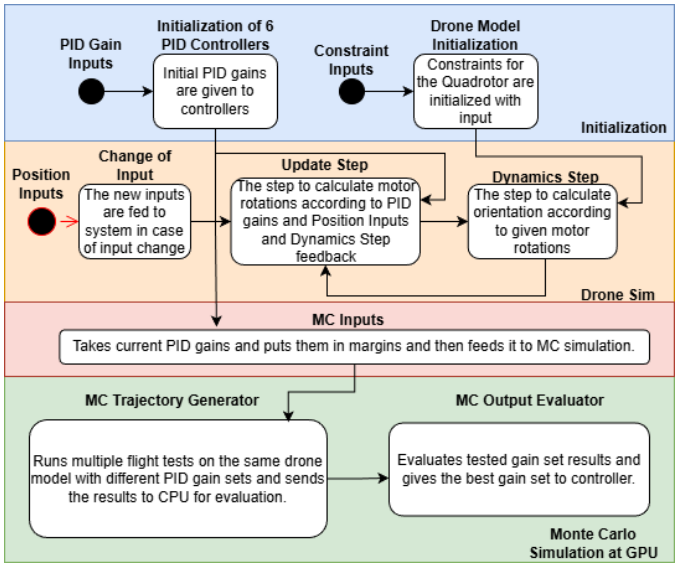

Fig. 1. A simplified representation of the system

work Designing a Self-Tuning PID Controller for AR Drone Quadrotor [4] uses a PID as a controller for the quadrotor and tunes the PID controller with a gradient-descent algorithm during flight. The similarity of this project comes from the idea of tuning quadrotor controller during flight but the difference is the tuning method where our work performs tuning with GPU accelerated Monte Carlo simulation while this work performs it with a gradient-descent algorithm. Gradient-descent algorithm may execute faster but it may converge to a local optima, it is sensitive to learning rate and it does not explore the full parameter space like our Monte Carlo simulation method.

Motor-Quadrotor Simulations Based on FOC and Cuckoo PID Tuning [5] is another PID controller based quadrotor simulation work where the parameters are tuned with Cuckoo Optimization Algorithm (COA) minimizes the squared-error cost functions considering three phases. Quadrotor motion dynamics, motor speed dynamics, and motor current dynamics. This work presents a more detailed drone simulation in comparison to ours where it works as a PC simulation as it has a lot more resources to use. This work is not designed for an embedded computer and be computationally expensive for it, where one of our key concepts is optimizing on the flight.

Modelling, Simulation, and Implementation of PID Controller on Quadrotors [6] work uses a drone dynamical model similar to ours and designs a PID controller for it. The work contains both simulation and implementation of the algorithm which is also in line with our aim. Where in this work the PID design is done manually and proceeds to the test the algorithm in simulation. This can be time consuming as there are other automated optimization methods such as the one we present.

A High-Performance Monte Carlo Simulation Toolbox for Uncertainty Quantification of Closed-Loop Systems [7] presents a parallelized Monte Carlo simulation toolbox implemented in C for shared-memory architectures. The core idea of this paper is the same, use massively parallel Monte

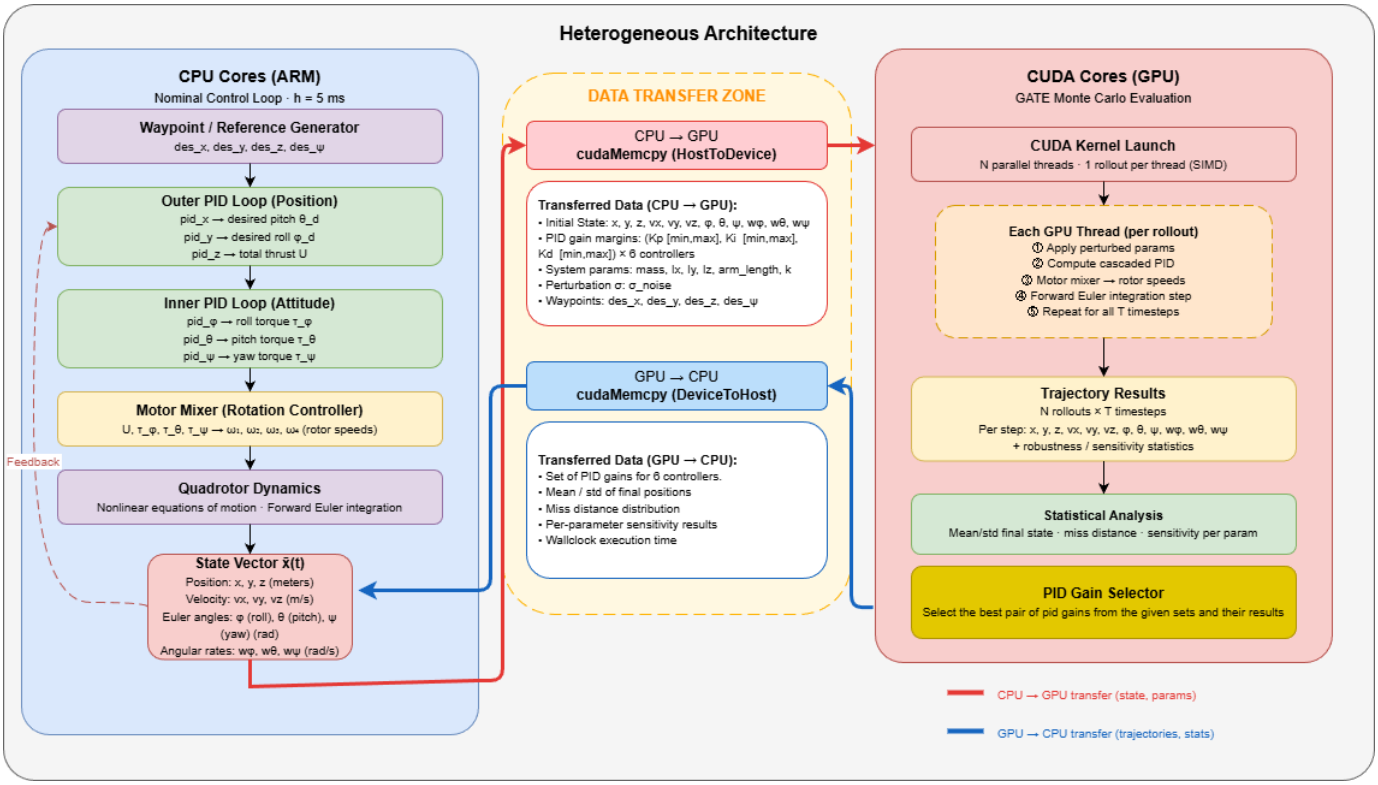

Fig. 2. More detailed representation of the system

Carlo simulation for controller (PID) tuning and performance quantification. It also demonstrates automatic PID tuning through Monte Carlo search which is another goal that we have. However, the application domain is a bioreactor not a quadrotor and this project runs on a high-performance workstation not on an embedded platform. Also, this work presents a CPU-only parallelism where we try to do it on GPU architecture.

Dynamic Response Optimization for Quadrotor UAVs Using a Parameter Self-Tuning Fuzzy PID Control Method [8] also focuses on PID gain adjustment where it presents a light-weight fuzzy logic rule set for PID gain optimization. It also addresses limitations of fixed PID gains and uses a simulation based evaluation. However, even though the fuzzy logic algorithm is light-weight and suitable for on board deployment, the performance depends heavily on the quality of hand-crafted fuzzy rules and the system does not explore the gain space for optimality.

Intelligent Controller Design for Quad-Rotor Stabilization in Presence of Parameter Variations [9] is another work that present self-tuning PID controller for quadrotors and its strengths and weaknesses are similar. This work is validated on a well-known physics simulator Gazebo and on real hardware with several experiments. Its success relies heavily on hand crafted fuzzy rules which requires a lot of expertise on the system and it is still not guaranteed to be suitable for many applications.

Fast Monte Carlo Analysis for 6-DoF Powered-Descent Guidance via GPU-Accelerated Sequential Convex Programming [10] introduces a GPU-accelerated Monte Carlo framework for a powered-descent trajectory optimization for rockets. This paper presents a heavy algorithm that needs a high-performance desktop/server GPU's and the controller structure is different from ours. However, even though the application field and the hardware is different, the core methodology of GPU-accelerated Monte Carlo simulation is the same. The implementation presented in this paper can be a guidance and inspiration for our work while we implement our own methodologies.

TABLE I

Work	Year	PID Controller	Tuning Automation	Embedded Device	Validation with Simulation	GPU Usage
Self-Tuning PID AR Drone [4]	2017	✓	✓	✓	✓	
FOC + Cuckoo PID [5]	2024	✓	✓		✓	
Modelling and PID on Quad [6]	2021	✓		✓	✓	
MC Toolbox Closed-Loop [7]	2022	✓	✓		✓	
Fuzzy PID Quad UAV [8]	2024	✓	✓		✓	
Intelligent Ctrl Quad [9]	2017	✓	✓	✓	✓	
GPU MC 6-DoF Descent [10]	2024		✓		✓	✓
This Work	2026	✓	✓	✓	✓	✓

III. Problem Definition

This section formally defines the quadrotor simulation and controller evaluation problem, including the system description, objectives, and constraints for the embedded heterogeneous platform.

A. System / Application Description

The application scenario consists of a quadrotor UAV simulation running on an NVIDIA Jetson embedded platform, where the CPU handles the nominal flight control loop and the GPU accelerates Monte Carlo-based controller robustness evaluation using the GATE framework architecture.

The quadrotor is modeled as a rigid body with four rotors generating thrust and torque in the body frame. The system state is defined as:

$$\bar{x} = [x, y, z, v_x, v_y, v_z, \phi, \theta, \psi, w_\phi, w_\theta, w_\psi]^T \tag{1}$$

where x, y, z denote the position of the center of mass in the inertial frame, v_x, v_y, v_z are the corresponding translational velocities, ϕ (roll), θ (pitch), and ψ (yaw) are the Euler attitude angles, and w_ϕ, w_θ, w_ψ are the angular velocities about the body-frame axes.

Each rotor i generates a force proportional to the square of its angular velocity:

$$f_i = k \cdot \omega_i^2, \quad i \in \{1, 2, 3, 4\} \tag{2}$$

where k is the propeller constant. The total thrust and torques are computed from the individual rotor forces as:

$$U = f_1 + f_2 + f_3 + f_4 \tag{3}$$
$$\tau_\psi = (f_1 + f_3 - f_2 - f_4) \cdot L \tag{4}$$
$$\tau_\theta = (f_1 - f_3) \cdot L \tag{5}$$
$$\tau_\phi = (f_2 - f_4) \cdot L \tag{6}$$

where L is the arm length from the center of mass to each rotor.

The translational accelerations in the inertial frame are:

$$a_x = \frac{U}{m}(\sin\psi \sin\phi + \cos\psi \sin\theta \cos\phi) \tag{7}$$
$$a_y = \frac{U}{m}(-\cos\psi \sin\phi + \sin\psi \sin\theta \cos\phi) \tag{8}$$
$$a_z = \frac{U}{m}(\cos\theta \cos\phi) - g \tag{9}$$

where m is the quadrotor mass and $g = 9.8$ m/s^2 is gravitational acceleration. The angular accelerations are governed by the Euler rotational equations:

$$\ddot{\psi} = \frac{(I_x + I_z - I_y) \cdot w_\phi \cdot w_\theta + \tau_\psi}{I_z} \tag{10}$$
$$\ddot{\theta} = \frac{(I_z - I_x - I_y) \cdot w_\psi \cdot w_\phi + \tau_\theta}{I_y} \tag{11}$$
$$\ddot{\phi} = \frac{(I_x + I_y - I_z) \cdot w_\psi \cdot w_\theta + \tau_\phi}{I_x} \tag{12}$$

where I_x, I_y, I_z are the principal moments of inertia. The state is integrated forward in time using the forward Euler method with a fixed step size $h = 0.005$ s:

$$\bar{x}(t + h) = \bar{x}(t) + h \cdot \dot{\bar{x}}(t) \tag{13}$$

The nominal system parameters used in this work are: $m = 1.4$ kg, $I_x = I_y = 0.0116$ kg·m^2, $I_z = 0.0232$ kg·m^2, $L = 1.0$ m, and $k = 10^{-6}$.

The controller follows a cascaded PID structure with an outer position loop and an inner attitude loop, reflecting the standard architecture used in real-world flight controllers. The outer position loop uses three PID controllers to generate the desired attitude references and total thrust from position error:

$$U = m \cdot g + \text{PID}_z(z_{des} - z) \tag{14}$$
$$\theta_d = \text{PID}_x(x_{des} - x) \tag{15}$$
$$\phi_d = -\text{PID}_y(y_{des} - y) \tag{16}$$

The desired angles are clamped to ± 0.44 rad ($\approx 25°$) to maintain validity of the small-angle approximation. The inner attitude loop uses three PID controllers to generate torque commands from attitude error:

$$\tau_\phi = \text{PID}_\phi(\phi_d - \phi) \tag{17}$$
$$\tau_\theta = \text{PID}_\theta(\theta_d - \theta) \tag{18}$$
$$\tau_\psi = \text{PID}_\psi(\psi_{des} - \psi) \tag{19}$$

Each PID controller implements derivative-on-measurement and anti-windup:

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau)\, d\tau - K_d \frac{d(\text{measured})}{dt} \tag{20}$$

where the derivative term acts on the measured value rather than the error to avoid derivative kick on setpoint changes, and the integral term is clamped to a configurable limit to prevent windup. Controller gains are selected based on target natural frequencies to ensure all closed-loop poles satisfy the Euler stability criterion $|1 + h \cdot s| < 1$. The inner attitude loop targets $\omega_n = 40$ rad/s and the outer position loop targets $\omega_n = 1.5$ rad/s, providing a bandwidth separation ratio of approximately 20:1 for cascade stability.

The motor mixer maps the total thrust and torques to individual rotor forces:

$$f_1 = \frac{U}{4} + \frac{\tau_\psi}{4L} + \frac{\tau_\theta}{2L} \tag{21}$$
$$f_2 = \frac{U}{4} - \frac{\tau_\psi}{4L} + \frac{\tau_\phi}{2L} \tag{22}$$
$$f_3 = \frac{U}{4} + \frac{\tau_\psi}{4L} - \frac{\tau_\theta}{2L} \tag{23}$$
$$f_4 = \frac{U}{4} - \frac{\tau_\psi}{4L} - \frac{\tau_\phi}{2L} \tag{24}$$

Each force is clamped to zero from below (motor saturation), and rotor speeds are obtained as $\omega_i = \sqrt{f_i / k}$.

For the offloaded evaluation, each candidate controller is represented by an 18-dimensional PID gain vector (six PID controllers, each with K_p, K_i, K_d):

$$K_j = \begin{bmatrix} K_{p,x} & K_{i,x} & K_{d,x} \\ K_{p,y} & K_{i,y} & K_{d,y} \\ K_{p,z} & K_{i,z} & K_{d,z} \\ K_{p,\phi} & K_{i,\phi} & K_{d,\phi} \\ K_{p,\theta} & K_{i,\theta} & K_{d,\theta} \\ K_{p,\psi} & K_{i,\psi} & K_{d,\psi} \end{bmatrix}^T_j . \tag{25}$$

The host samples N candidates around an initial, manually chosen gain vector using multiplicative random sampling,

$$K_j = K_{base} \odot r_j, \qquad r_j \sim \mathcal{N}(1, \sigma_K^2), \tag{26}$$

where $\odot$ is element-wise multiplication and σ_K sets the spread of the search. Each rollout produces a trajectory $\bar{x}_j(t)$ for $t \in [0, T]$ using the same dynamics, and is scored by a cost combining position RMSE, settling time, and overshoot:

$$\text{RMSE}_j = \sqrt{\tfrac{1}{M} \textstyle\sum_{n=1}^{M} \|p_{j,n} - p_{ref,n}\|_2^2}, \tag{27}$$

$$J_j = \alpha\, \text{RMSE}_j + \beta\, T_{s,j} + \gamma\, O_j, \tag{28}$$

where $T_{s,j}$ is the settling time, O_j the overshoot penalty, and (α, β, γ) weighting coefficients. The host then selects $j^\star = \arg\min_j J_j$ (excluding crashed rollouts) and updates the controller with $K^\star = K_{j^\star}$. Diverging or NaN trajectories are detected and penalized so they can never be selected.

The key assumptions of the system are: the quadrotor operates in a simulation environment without real sensor hardware; low-level aerodynamic effects such as blade flapping and ground effect are neglected; environmental perturbations are disabled in the current implementation so that the effect of gain selection can be isolated; candidate gains are sampled from a normal distribution around the initial gains; and the chosen fixed-step integrators are sufficient for the dynamics at the selected step size.

B. Objectives

The objectives of this project are as follows:

- Demonstrate that the GATE GPU-parallel Monte Carlo architecture can be deployed on a resource-constrained NVIDIA Jetson embedded platform for controller robustness evaluation.
- Maintain real-time execution of the CPU-side cascaded PID control loop within the 5 ms step size on the Jetson's ARM cores.
- Maximize GPU-side Monte Carlo evaluation throughput by running N parallel trajectory rollouts simultaneously using CUDA.
- Enable PID gain selection by evaluating multiple candidate gain sets across Monte Carlo scenarios on the GPU and feeding the best-performing set back to the CPU controller.
- Provide quantitative performance comparisons between CPU-only sequential evaluation and CPU+GPU parallel evaluation, measuring speedup ratio and scalability with increasing rollout count.
- Measure and characterize the CPU–GPU data transfer overhead to understand its impact on total evaluation latency.

C. Constraints

The system operates under the following constraints:

- **Timing:** The CPU control loop must complete one full iteration (state read, six PID computations, motor mixer, Euler integration) within 5 ms. Violation of this deadline results in simulation timing inaccuracy.
- **Hardware:** The system runs on a single NVIDIA Jetson board with shared memory between the ARM CPU and the integrated GPU. Both processors share the same power and thermal envelope.
- **Memory:** GPU device memory must accommodate N rollouts of perturbation data ($N \times T$ values per perturbation source) and $N \times T \times 12$ state trajectory output values simultaneously. The Jetson's GPU memory is significantly smaller than desktop-class GPUs, which limits the maximum N and T.
- **Data transfer:** All perturbation data and system parameters must be transferred from host (CPU) memory to device (GPU) memory via `cudaMemcpy` before kernel launch. Result trajectories must be transferred back after kernel completion. This transfer latency is part of the total evaluation time.
- **Numerical stability:** The forward Euler integrator can result in exponentially growing results where eventually all calculations converge to NaN. This constrains the maximum PID gains that can be used without causing divergence.
- **Software:** The system runs a minimal Linux image with CUDA toolkit on the Jetson. No real-time operating system (RTOS) is used; timing guarantees are best-effort under standard Linux scheduling.

IV. System Architecture and Design

A. Hardware/Software Architecture

The hardware will be a NVIDIA Jetson Nano development board. The programmable units of it are Integrated 128-core Maxwell GPU and Quad-core ARM Cortex-A57 processor clocked at 1.43 GHz [11].

The schematic representation of the interaction between these two computational units is given in Figure 2. The system will run drone simulation, communicating with monitor tasks on CPU. Where the drone dynamics and controller algorithm runs at a specific time step. The GPU will run Monte Carlo simulations to test the performance of different controllers on a quadrotor model with constant parameters and give the results to CPU. The CPU is also responsible for starting the Monte Carlo simulation on GPU via feeding it the data it requires to operate and then update its controller from the data acquired from GPU when the Monte Carlo test is finished.

B. Dataflow and System Model

As shown in the Figure 1 the system steps are initialization, drone simulation, Monte Carlo simulation start, Monte Carlo simulation end and Data transfer and logging. The initialization part is self explanatory where the initial PID gains and system constraints such as time step or mechanical constraints defining the quadrotor are fed into the model. Drone simulation part runs periodically and responds to the desired position inputs coming from the monitoring device. The drone simulation part consists of two main parts which are update step and dynamics step.

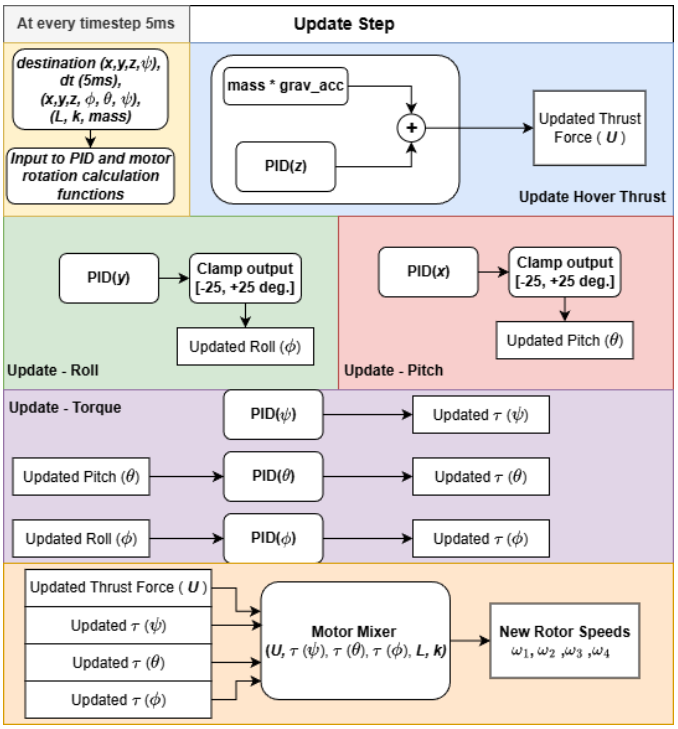

Fig. 3. Schematic of Update Step

Update step part is responsible for calculating the PID controller outputs (which are the new rotor speeds) and feed them to the dynamics part. Illustration of this is given in Figure 3.

Update step part runs 6 PID controller calculations in order to generate the required rotor speed outputs. The algorithm of these PID controllers are the same where the only differences between them are the PID gains. The schematic representing the flow is given in Figure 4.

The Update Step also include a Motor Mixer part where in this the required rotations for each rotor are calculated separately. The schematic representing the calculation is given in Figure 5.

The Dynamics Step has three phases, calculation of angular and directional accelerations at time t, calculation of the velocities at time t + 1 according to accelerations and calculation of the orientation at time t + 1 according to velocities at time t. The acceleration calculation is is done by implementing

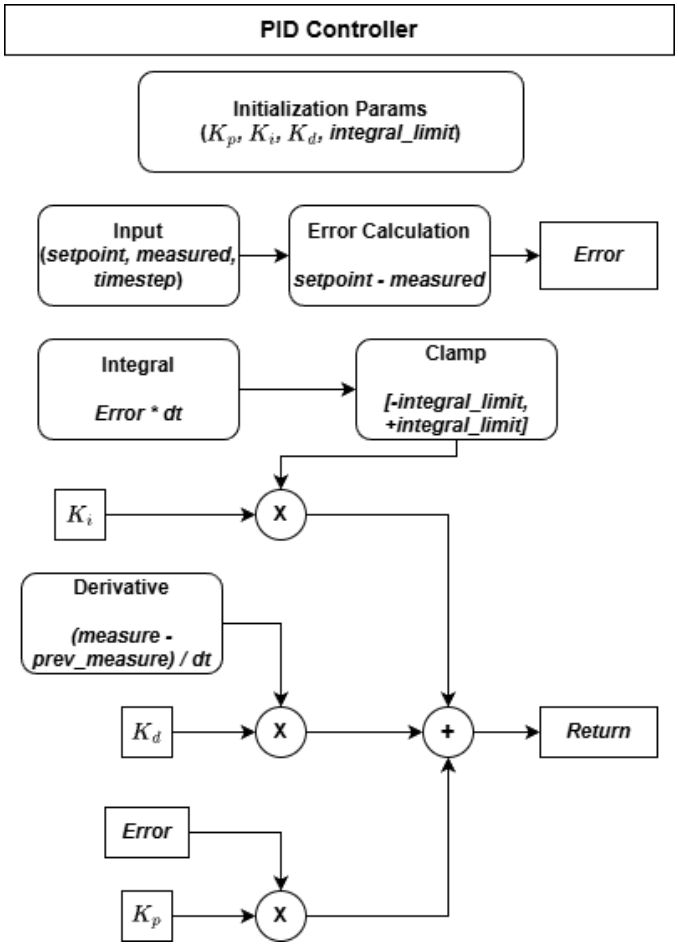

Fig. 4. PID Calculation Schematic

our drone dynamics model where the required values are initialized as zero, velocity and orientation part is done by Euler's numerical method. The Euler's numerical method is defined as following.

Given the initial value problem:

$$f(x, y) = \frac{dy}{dx} =, \quad y(x_0) = y_0 \tag{29}$$

The iterative update formula is:

$$y_{n+1} = y_n + h \cdot f(x_n, y_n) \tag{30}$$

where:

$$x_{n+1} = x_n + h \tag{31}$$

$$h = \text{step size} \tag{32}$$

$$n = 0, 1, 2, \ldots \tag{33}$$

The dynamics calculations follow this logic and the implementation of them are illustrated in the figures 6 7 8.

C. Requirements Analysis

1) Functional Requirements: FR-01 — Quadrotor Dynamic Model Simulation The system shall simulate a 6-DoF quadrotor dynamic model on the Jetson CPU, computing translational

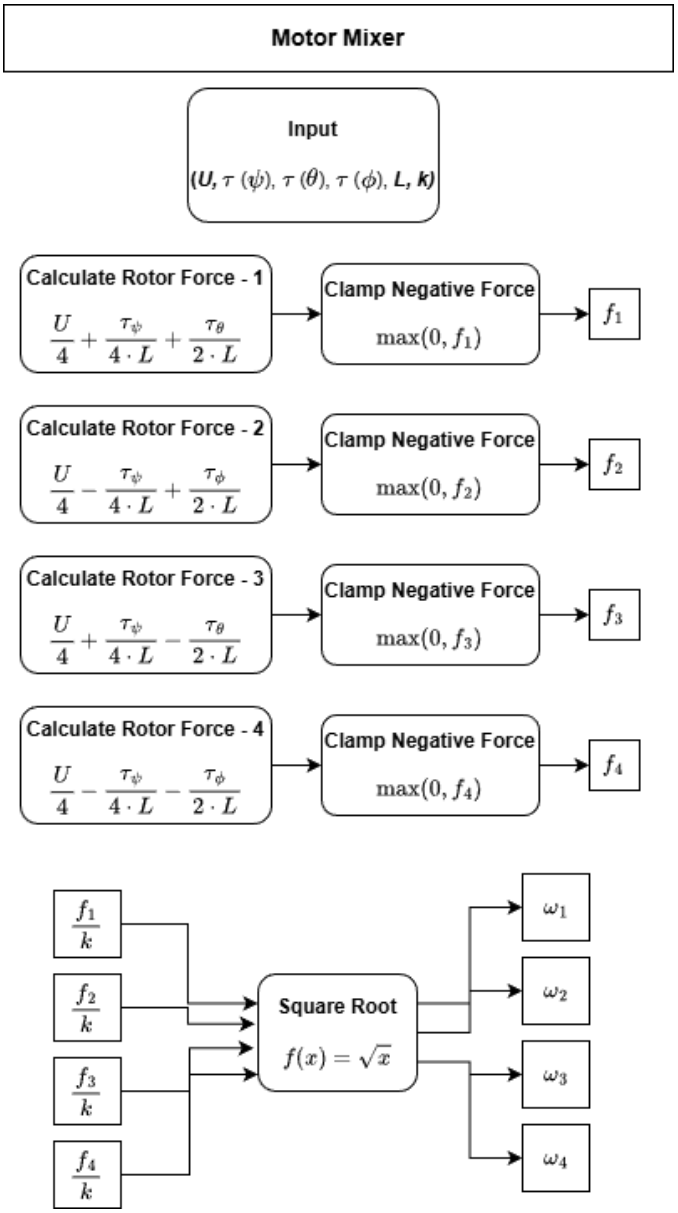

Fig. 5. Motor Mixer Schematic

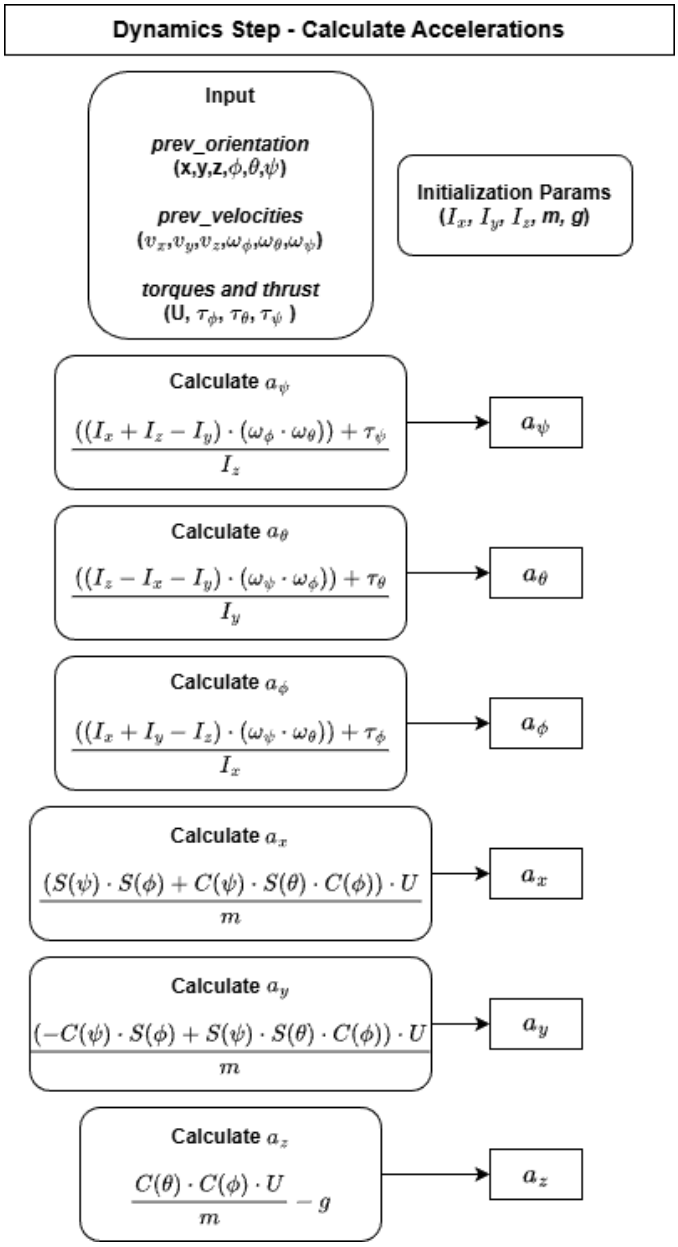

Fig. 6. Dynamics Step - Acceleration Schematic

and rotational states (position, velocity, orientation, angular rates) at each integration timestep.

FR-02 — Six-Channel PID Control The system shall implement six independent PID controllers governing roll, pitch, yaw, and their respective angular rates, each with configurable gains (Kp, Ki, Kd).

FR-03 — PID Gain Parameterization The system shall accept a gain vector of 18 parameters (Kp, Ki, Kd for each of the six controllers) as input to each simulation run, with user-definable upper and lower bounds per parameter.

FR-04 — Monte Carlo Sample Generation The system shall generate N candidate PID gain vectors by sampling from Gaussian distribution within the given bounds on the GPU.

FR-05 — GPU-Parallel Simulation Execution The system shall execute N independent closed-loop simulations in parallel on the Jetson GPU, where each simulation integrates the quadrotor model with a distinct candidate gain vector and a given command trajectory.

FR-06 — Optimal Gain Selection The system shall identify and return the PID gain vector that minimizes the cost function across all N Monte Carlo samples, along with its cost value.

FR-07 — Statistical Summary Output The system shall compute and report summary statistics of the Monte Carlo tests.

FR-08 — Real-Time and Fast Modes The system shall support both real time simulation and full-speed modes. Where in the latter system behavior will be observed from logs.

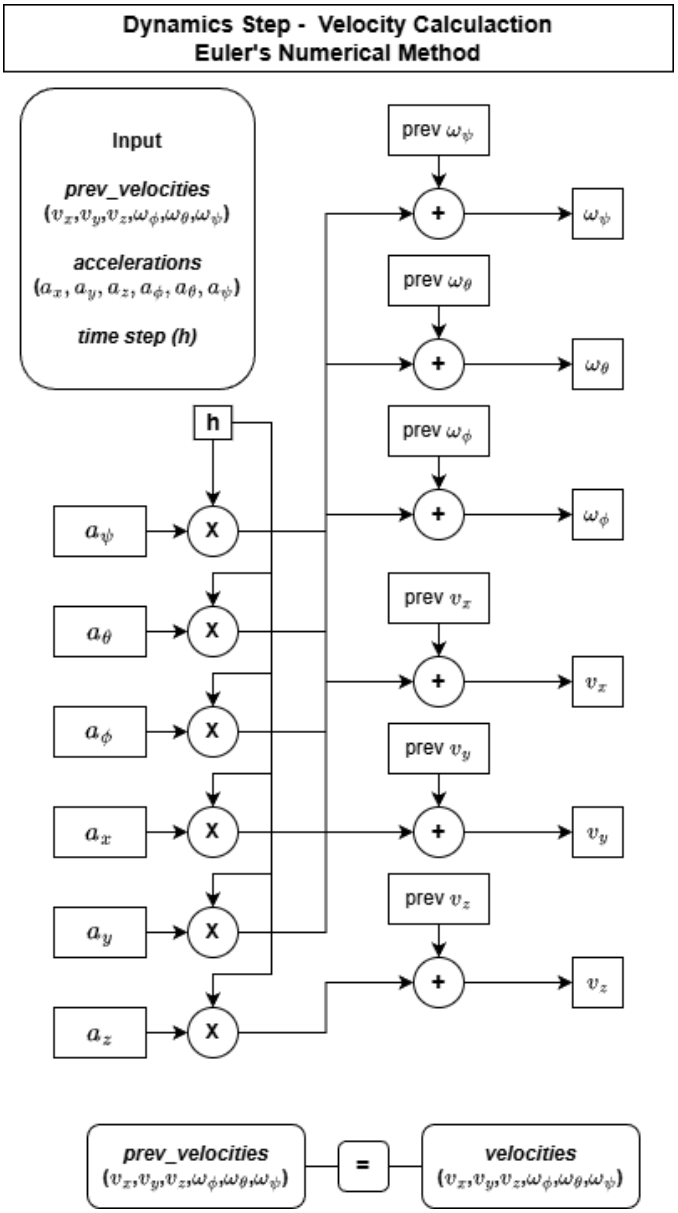

Fig. 7. Dynamics Step - Velocities Schematic

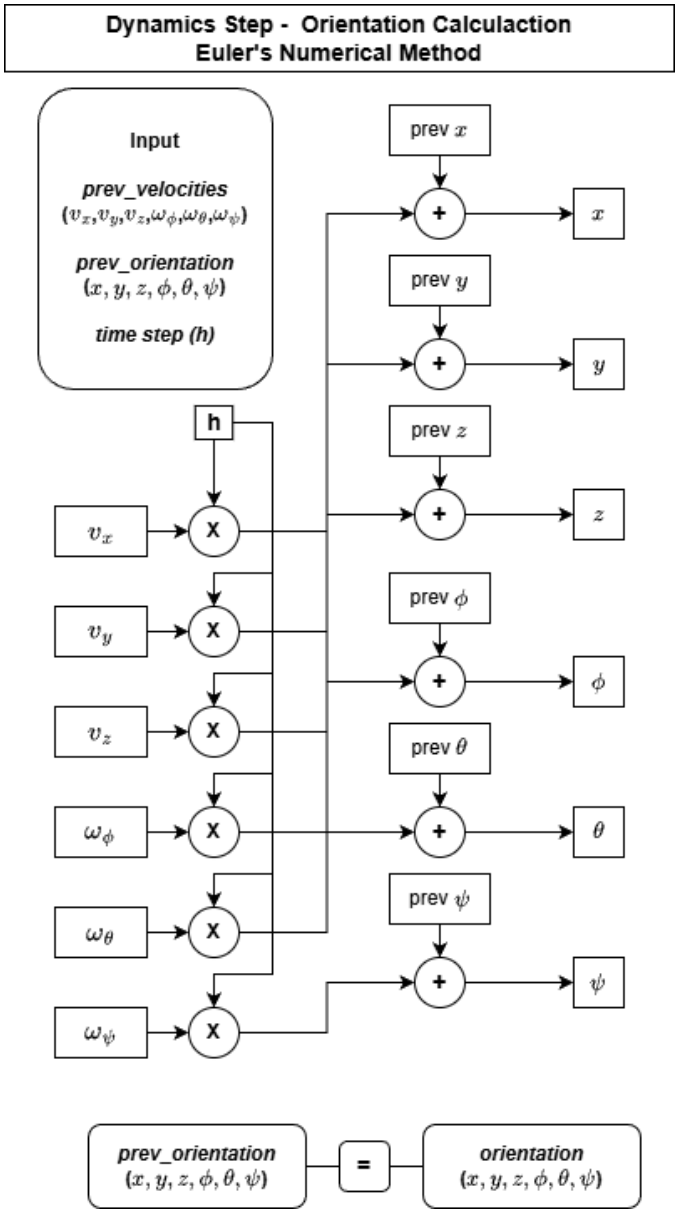

Fig. 8. Dynamics Step - Orientation Schematic

FR-09 — Result Logging and Export The system shall log the full set of evaluated gain vectors and their corresponding costs to persistent storage, in a format suitable for post-processing (CSV or binary).

FR-10 — Simulation Timestep Configuration The system shall allow the user to configure the integration timestep (dt) and the total simulation horizon (T) for each Monte Carlo run.

FR-11 — Task Completion Before Time-step All real-time tasks should finish before the desired time-step in real-time.

2) Non-Functional Requirements: NFR-01 — Monte Carlo Throughput The system shall evaluate at least 1,000 complete closed-loop simulations per second on the target Jetson GPU.

NFR-02 — Memory Footprint per Simulation Thread Each GPU thread shall consume no more than 2 KB of local memory (registers + local variables), excluding shared memory.

NFR-03 — Numerical Stability The quadrotor simulation shall remain numerically stable (no NaN or Inf states) for all PID gain combinations within the user-defined parameter bounds, using a fixed-step integrator with dt.

NFR-04 — Result Determinism Given identical input parameters and randomized parameters, the system shall produce identical results across repeated runs on the same Jetson hardware.

NFR-05 — Latency from Command to Optimized Gains The end-to-end latency from receiving a new command trajectory to outputting the optimized PID gain vector shall not exceed 15 seconds.

NFR-06 — Scalability with Sample Count The system's

execution time shall scale linearly with the number of Monte Carlo samples N, for N between 1,000 and 50,000.

NFR-07 — Maintainability and Code Quality The code shall be structured with separation between the dynamic model, PID controller, cost function, and Monte Carlo harness, allowing any component to be modified independently.

D. Timing and Execution Model

The task execution model of our proposed system is both periodic and event-driven. Periodically executing part of the system is real-time simulation of the drone dynamics and PID controller calculations. Event-driven parts start with desired position and rotation info coming from the remote monitor. This new position task starts the GPU driven Monte Carlo evaluation cycle where the system tries to find the best possible PID gain set for this task.

Timing is cruical for such a simulation and testing system as the simulation is expected to run dynamics and control part faster than the actual time step in real time. Also if such an architecture is desired to be used on a real quadrotor as its flight optimizer the Monte Carlo testing should not take too long to finish as the moving task is not executed, or even though it is executed it should not delay the outputs of the optimized controller.

E. Model of Computation

The selected model of computation is Hybrid Automata. The behavior of the system consists of continuous dynamics and discrete computations. Here continuous dynamics includes the continuous change of the drone's orientation, velocities, and accelerations. Discrete computation includes the sample driven calculations. updates occurring at discrete periodic instances and the state transitions happening in the system such as system initialization, dynamics calculation step, control step and logging.

F. Assumptions and Design Limitations

The performance of this PID optimizer and drone simulator CPU-GPU heterogeneous architecture is assumed to be similar on a real drone on ideal environmental factors as the simulated environment does not include harsh conditions as the limited computational power on this embedded device prevents such detailed representations of real world.

Also, limited number of CUDA cores is another obstacle in finding the best possible controller for the given quadrotor in short time. This may result in finding PID gain sets in local minima in order to finish evaluation task in a short time.

G. Resource Model

The Jetson boards share a unified memory architecture — CPU and GPU access the same physical DRAM. This is both an advantage and a constraint. On the advantage side, PCIe transfer is avoided. On the constraint side, quadrotor simulation state, PID controller buffers, and the Monte Carlo sample arrays all compete for the same pool.

Each quadrotor state vector and each controller will need its own space. On a single instance, this is a trivially small size. However, the GPU Monte Carlo side is where memory matters. If you run N parallel simulations, each needing its own copy of the quadrotor state plus the PID gain set being evaluated, memory scales linearly with N. This can result in short evaluation intervals or partitioning the Monte Carlo runs into chunks to fit in available memory.

V. Methodology

A. Implementation Details

The implemented system is a heterogeneous CPU–GPU application. From the embedded-host viewpoint of this report, the CPU is responsible for everything that must happen in real time or that coordinates the system, and the GPU is an offloaded coprocessor that the CPU drives. We describe the host-side components first — in the detail appropriate to a CPU-focused report — and then summarize the offloaded accelerator, distinguishing our own code from reused libraries.

Quadrotor Dynamics Simulator (CPU, custom): The quadrotor is modeled as a rigid body with four rotors, each generating thrust $F = k \cdot \omega^2$. The nonlinear equations of motion compute translational accelerations through Euler-angle rotation and rotational accelerations through the gyroscopic cross-coupling and inertia terms, following the standard formulation also used in [2]. The nominal host loop integrates the 12-dimensional state with *forward Euler* at $h = 0.005\,\mathrm{s}$, chosen with the real-time computational-complexity considerations of [3] in mind: forward Euler is the cheapest integrator that remains stable at this step, which matters because the loop must finish a full state-read, six PID computations, the motor mixer, and the integration inside the 5 ms budget. The simulator is written from scratch in C++.

Cascaded PID Controller (CPU, custom): The controller has two layers, both running on the host. The outer position loop uses `pid_x`, `pid_y`, `pid_z`: `pid_z` adds a thrust correction to the hover feedforward ($m \cdot g$), while `pid_x` and `pid_y` produce desired pitch (θ_d) and roll (ϕ_d), clamped to $\pm 25^\circ$ to keep the small-angle approximation valid. The inner attitude loop (`pid_`ϕ, `pid_`θ, `pid_`ψ) produces body torques. All six controllers use derivative-on-measurement (computing $-d(\text{measured})/dt$ rather than $d(\text{error})/dt$) to avoid derivative kick on setpoint changes, and a clamped integral for anti-windup. A motor mixer inverts the force/torque allocation into four rotor speeds $\omega_1 \ldots \omega_4$, with negative thrust clamped to zero. The controller is written from scratch in C++.

Reference / Waypoint Generator (CPU, custom): The host generates a smooth four-phase reference mission (Algorithm 2) using a SmoothStep interpolation. This avoids the unrealistic square-wave setpoints that would otherwise inject step discontinuities into the controller and unfairly penalize candidates; the mission waypoints are read from the configuration file so different flights can be requested without recompiling.

Host Orchestration and Best-Gain Selection (CPU, custom): This is the part that makes the CPU the system coordinator and is central to the embedded design. The top-level driver (Algorithm 1) loads the configuration, samples N candidate gain

vectors around the base controller, copies them to the device, launches the two offloaded kernels and synchronizes, copies back the per-rollout costs, and then performs the $\arg\min$ *on the CPU* over the non-crashed candidates to choose the best gain set. The selected gains update the cascaded controller. Crucially, this offload is advisory and is issued between control periods, so it never blocks the real-time loop: the only datum that flows back into the loop is a small gain vector, exchanged at a period boundary.

CPU Visualization / Validation Simulator (CPU, custom, using GLFW + Python): For validation the host re-runs the winning gain set with forward Euler (Algorithm 5) and drives a GLFW/OpenGL 3D viewer one timestep at a time; the 2D per-axis tracking plots are produced with Python/Matplotlib from the logged CSV. This host-side replay is what produces the trajectory figures in Section VII.

Offloaded GPU Accelerator (re-engineered GATE): The expensive part — evaluating many candidate controllers — is offloaded to the GPU. We re-engineer the GATE framework: its generate–simulate–evaluate–select structure is reused, but its environmental-perturbation modules are removed and the focus is changed from perturbation robustness to PID gain comparison, as summarized in Fig. 9. The accelerator runs two kernels: a rollout kernel (one thread per candidate, RK4 integration — the GATE default) and a scoring kernel (crash detection plus the cost of Eq. (28)). From the host's perspective these are launched, synchronized, and harvested for compact results; their internal optimization is the subject of the companion parallel-computing report and is not detailed here.

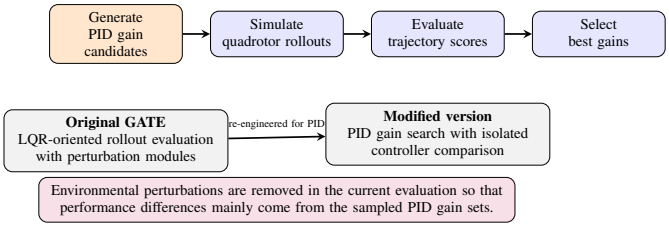

Fig. 9. Modified GATE pipeline for GPU-based PID gain evaluation.

Custom vs. reused: **Custom (host):** dynamics simulator, cascaded PID, motor mixer, reference generator, top-level orchestration driver, best-gain selection, CPU validation simulator, logging, and integration. **Custom (offloaded):** modified GATE pipeline, rollout kernel, scoring kernel, candidate handling. **Reused:** CUDA runtime API and GPU execution model (NVIDIA), the GATE framework structure (including its RK4 integrator), Eigen (v3.3.8), GLFW (v3.4) for 3D animation, and Python/Matplotlib for 2D plots.

B. Algorithm Description

The system is formalized in five algorithms. Three run entirely on the host — the top-level driver (Algorithm 1), the reference generator (Algorithm 2), and the validation simulator (Algorithm 5); the remaining two (Algorithms 3 and 4) are the offloaded kernels the host launches. We present all five for completeness, but the emphasis is on how the host orchestrates the workload.

Algorithm 1 MAIN – Top-level driver running on the CPU

cfg ← LOADCONFIG("`config.txt`") ▷ base gains, N_{active}, σ, seed
(z_1, x_t, y_t, z_2) ← PROMPTUSER( )
$\mathcal{M}$ ← REFERENCETRAJECTORY(z_1, x_t, y_t, z_2)
INITGATEPIPELINE($\mathcal{M}$) ▷ dynamics, GATE buffers
// Sample per-rollout PID gains
$G[0]$ ← *cfg.base_gains*
for $r = 1$ **to** $N_{\text{active}} - 1$ **do**
  for each PID gain g **do**
    $\alpha \sim \mathcal{N}(1, \sigma)$; $\alpha \leftarrow \max(\alpha, 0.05)$
    $G[r].g$ ← *cfg.base_gains*$.g \times \alpha$
  end for
end for
COPYTODEVICE($G \rightarrow$ *pid_gains_d*); INITDEVICEPIDSTATES( )
// Launch CUDA kernels: 1 thread = 1 rollout
ROLLOUTKERNEL( ); CUDADEVICESYNCHRONIZE( )
SCORINGKERNEL( ); CUDAMEMCPY(*costs_d* $\rightarrow$ *costs_h*)
// Pick lowest-cost rollout on the CPU
$r^\star \leftarrow \arg\min_{r:\, crashed[r]=0}$ *costs_h.combined*$[r]$
best_traj ← *trajectories_h*$[r^\star]$; *best_gains* ← $G[r^\star]$
SAVECSV("`best_trajectory.csv`", *best_traj*, $\mathcal{M}$)
cpu_sim ← CPUSIMULATOR(*best_gains*, $\mathcal{M}$)
LAUNCHVIEWER(*best_traj*, *cpu_sim*, $\mathcal{M}$)

The host PID routine — used both on the host and, in identical form, inside the rollout kernel — implements $u = K_p e + K_i \int e\,dt + K_d(-d(\text{meas})/dt)$ with derivative-on-measurement and a clamped integral. Note the deliberate integrator split: the host loop and validation simulator use forward Euler (cheap, real-time), while the offloaded rollouts use RK4 (more accurate, the GATE default), which is acceptable because the offload is not on the real-time path.

C. Platform and Tools

The system is *designed* for a heterogeneous CPU+GPU embedded platform such as an NVIDIA Jetson board, which provides ARM CPU cores and a CUDA-capable GPU on a single unified-memory SoC; the candidate device is the Jetson Nano (integrated 128-core Maxwell GPU, quad-core ARM Cortex-A57 at 1.43 GHz [11]). The host architecture, the advisory offload, and the compact per-thread budget were chosen so the implementation can fit such a device. The platform on the board is built by NVIDIA SDK Manager with JetPack 4.5.1. The system comes with gcc 7.5, which is sufficient as the code requires C++ 14, and the CUDA 10.2 environment is installed later, which is also needed.

The desktop simulation experiments reported here were carried out on a Windows PC with a discrete NVIDIA GPU. The software is written in C++ (C++14) with CUDA C/C++, built in Visual Studio 2019 (toolset v142, CUDA v10.2).

Algorithm 2 REFERENCETRAJECTORY$(k, \mathcal{M})$ – smooth four point trajectory (CPU)

Require: timestep index k; mission $\mathcal{M} = (z_1, x_t, y_t, z_2, k_1, k_2, k_3)$
Ensure: desired position (x^*, y^*, z^*)

function SMOOTHSTEP(p)
  if $p \le 0$ **then**
    return 0
  end if
  if $p \ge 1$ **then**
    return 1
  end if
  return $p^2(3 - 2p)$
end function

if $k < k_1$ **then**
  $s \leftarrow$ SMOOTHSTEP(k/k_1); $(x^*, y^*, z^*) \leftarrow (0,\ 0,\ sz_1)$
else if $k < k_2$ **then**
  $s \leftarrow$ SMOOTHSTEP($(k - k_1)/(k_2 - k_1)$); $(x^*, y^*, z^*) \leftarrow (sx_t,\ sy_t,\ z_1)$
else if $k < k_3$ **then**
  $s \leftarrow$ SMOOTHSTEP($(k - k_2)/(k_3 - k_2)$); $(x^*, y^*, z^*) \leftarrow (x_t,\ y_t,\ z_1 + s(z_2 - z_1))$
else
  $(x^*, y^*, z^*) \leftarrow (x_t, y_t, z_2)$
end if
return (x^*, y^*, z^*)

Algorithm 3 ROLLOUTKERNEL — offloaded GPU kernel; one thread = one rollout

(Each GPU thread executes this code independently.)
tid $\leftarrow$ `blockDim.x · blockIdx.x + threadIdx.x`
if *tid* $\ge N_{\text{rollouts}}$ **then return**
$\mathbf{x} \leftarrow$ *x0_d*[*tid*]; *gains* $\leftarrow$ *pid_gains_d*[*tid*]; *traj*[*tid*, 0] $\leftarrow \mathbf{x}$
for $k = 1$ **to** $T - 1$ **do**
  $(x^*, y^*, z^*) \leftarrow$ REFERENCETRAJECTORY$(k, \mathcal{M})$
  $U \leftarrow mg +$ PID$(z^*, x_z, \textit{gains.z})$
  $\theta_d \leftarrow$ PID$(x^*, x_x, \textit{gains.x})$; $\phi_d \leftarrow -$PID$(y^*, x_y, \textit{gains.y})$
  clamp ϕ_d, θ_d to ± 0.44
  $\tau_\phi \leftarrow$ PID$(\phi_d, x_\phi, \textit{gains.}\phi)$; $\tau_\theta \leftarrow$ PID$(\theta_d, x_\theta, \textit{gains.}\theta)$; $\tau_\psi \leftarrow$ PID$(0, x_\psi, \textit{gains.}\psi)$
  $\mathbf{u} \leftarrow (U, \tau_\phi, \tau_\theta, \tau_\psi)$
  // RK4 step on $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ *(GATE default integrator)*
  $\mathbf{k}_1 \leftarrow f(\mathbf{x}, \mathbf{u})$; $\mathbf{k}_2 \leftarrow f(\mathbf{x} + \frac{\Delta t}{2}\mathbf{k}_1, \mathbf{u})$; $\mathbf{k}_3 \leftarrow f(\mathbf{x} + \frac{\Delta t}{2}\mathbf{k}_2, \mathbf{u})$; $\mathbf{k}_4 \leftarrow f(\mathbf{x} + \Delta t\,\mathbf{k}_3, \mathbf{u})$
  $\mathbf{x} \leftarrow \mathbf{x} + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$; *traj*[*tid*, k] $\leftarrow \mathbf{x}$
end for

Algorithm 4 SCORINGKERNEL – offloaded GPU kernel; per-rollout cost

tid $\leftarrow$ `blockDim.x · blockIdx.x + threadIdx.x`; **if** *tid* $\ge N_{\text{rollouts}}$ **then return**
crashed $\leftarrow 0$; *sse* $\leftarrow 0$; *ov* $\leftarrow 0$; *last_out* $\leftarrow -1$
for $k = 0$ **to** $T - 1$ **do**
  extract (p, angles) from *traj*[k]
  if p out of range **or** angle extreme **or** NaN **then**
    crashed $\leftarrow 1$
  end if
  $(r_x, r_y, r_z) \leftarrow$ COMPUTEREFERENCE$(k, \mathcal{M})$; *sse* $\leftarrow$ *sse* $+ \|p - r\|_2^2$
  during motion phases, update overshoot *ov*
  if $k \ge k_3$ **and** $\|p - p_{\textit{target}}\|_2 > \textit{tol}$ **then** *last_out* $\leftarrow k$
end for
rmse $\leftarrow \sqrt{\textit{sse}/(3T)}$; *settle* $\leftarrow (\textit{last\_out} + 1 - k_3)\Delta t$ **if** *last_out* ≥ 0 **else** 0
combined $\leftarrow w_R$*rmse* $+ w_S$*settle* $+ w_O$*ov* $+ w_C$*crashed*

Reused libraries are GATE (which depends on Eigen v3.3.8 and supplies the RK4 integrator) and GLFW v3.4 for the 3D flight animation; the 2D plots use Python 3.11 with Matplotlib 3.10.9.

D. Datasets / Workloads

The system is entirely simulation-based. The nominal quadrotor parameters are $m = 1.4\,\text{kg}$, $L = 1.0\,\text{m}$, $I_x = I_y = 0.0116\,\text{kg·m}^2$, $I_z = 0.0232\,\text{kg·m}^2$, $k = 1 \times 10^{-6}$, $g = 9.8\,\text{m/s}^2$. The cost weights in Eq. (28) are $(\alpha, \beta, \gamma) = (1,\ 0.05,\ 0.5)$. The reference mission is the smooth four-phase trajectory of Algorithm 2 (a vertical climb, an xy-translation, and a final altitude change), with waypoints set from the configuration file. The tuning experiments use the mission $(0,0,0) \to (5,5,5) \to (5,5,2)$, and the literature comparison uses $(0,0,0) \to (2,2,2) \to (2,2,0)$. Each evaluation batch uses $N = 1024$ rollouts, with candidate gains sampled multiplicatively around the base vector at a user-selected variance σ_K and a fixed seed.

E. Evaluation Metrics

Consistent with the embedded-host focus, the evaluation combines host/system metrics with control quality. The **host/embedded metrics** are: real-time feasibility of the nominal control loop against the 5 ms (200 Hz) step; the host-observed **orchestration wall-clock** of one offloaded evaluation batch (the time the host waits between issuing the kernels and receiving the result); and the **per-thread memory footprint** of the offloaded kernels (registers, constant memory, total), measured at build time with PTXAS (`-Xptxas -v`) against a 2 KB-per-thread budget (NFR-02), since this footprint bounds how large N can be on a small device. The **control-performance metrics** are position RMSE, settling time, overshoot, the combined score of Eq. (28), and the crash count out of N. Finally, the tuned controller's per-axis tracking is compared against the simulation results reported by Doukhi et al. [9] and Babu et al. [4].

We note that several items planned in the proposal — a CPU-Serial vs. CPU-Parallel (OpenMP) vs. GPU-CUDA speedup study, on-device energy via `tegrastats`, and an on-Jetson scalability sweep — are *not* reported here, because

Algorithm 5 CPUSIMULATOR$(g, \mathcal{M})$ – validation/visualization on the host

Require: winning PID gains g and mission $\mathcal{M}$

$\mathbf{x} \leftarrow \mathbf{0}_{12}$; reset PID state; $k \leftarrow 0$
for $k = 0$ **to** $T - 1$ **do**
  $(x^*, y^*, z^*) \leftarrow$ COMPUTEREFERENCE$(k, \mathcal{M})$
  $\mathbf{u} \leftarrow$ CASCADEDPID$(\mathbf{x}, x^*, y^*, z^*, g)$
  $\mathbf{x} \leftarrow$ EULERSTEP$(\mathbf{x}, \mathbf{u}, \Delta t)$
end for

the experiments were run on a desktop rather than on the Jetson. These are stated explicitly in Sections VII-D and VIII.

VI. System-Level Considerations

Real-time constraints and latency: The host control loop is designed to run at 200 Hz (5 ms period) without interruption. The offloaded gain search runs asynchronously and never sits inside the loop: only the small selected gain vector flows back, and only between control periods, so the loop's deadline is structurally protected. On a Jetson the host–device transfer latency over shared memory would be the remaining factor; on the desktop test platform the host observes a full $N = 1024$ evaluation returning in roughly 0.2 s, but on-device latency has not been measured.

Energy efficiency: Power consumption of CPU-only versus offloaded execution matters on embedded platforms. Because the experiments ran on a desktop, on-device energy (e.g. via `tegrastats`) was not measured and remains future work; the energy-relevant design choices — a compact per-thread footprint and a bounded rollout count — are already in place.

Scalability: The workload scales with the number of rollouts N. On desktop GPUs, [2] showed strong scaling to thousands of rollouts; on a Jetson's smaller GPU the useful N is bounded by occupancy and memory, and the CPU-vs-GPU crossover is expected to differ. Characterizing this on-device is a remaining goal.

Reliability and fault tolerance: The offloaded result is advisory: if it fails, times out, or returns only crashed candidates, the host keeps the current gains. The scoring stage additionally guarantees that diverging/NaN trajectories are penalized and never selected, so a bad batch cannot push an unstable controller onto the host loop.

Security considerations: The system runs as a self-contained simulation with no external interfaces during operation, so security risks are minimal at this stage.

Quantitative results for the items measured on the test platform appear in Section VII; the on-device items are revisited in Sections VII-D and VIII.

VII. Results and Evaluation

This section presents and analyzes the results obtained from the implemented system. From the embedded-host perspective the evaluation answers three questions: does the host control loop and its orchestration behave as a real-time embedded workload should; does the offloaded search fit the resource budget of a Jetson-class device; and does the host, using that search, produce a usable controller. The first two are addressed with timing and a build-time footprint measurement, the third with a controller-tuning study and a literature comparison.

A. Desktop Simulation Performance Results

Experimental setup. All experiments ran on the desktop test platform described in the Platform and Tools subsection (Section V). The reference mission was $(0, 0, 0) \to (5, 5, 5) \to (5, 5, 2)$, every evaluation batch used $N = 1024$ rollouts, and the host re-simulated the winning gains for the figures (Algorithm 5). The 3D views are the GLFW host viewer; the per-axis plots are the Matplotlib output from the logged CSV.

Host-side timing. The host issues one offloaded batch and waits for the result between control periods. On the test platform a complete $N = 1024$ evaluation (sampling, both kernels, synchronization, and copy-back) returns to the host in roughly 0.2 s, and the three-stage search below completes in well under one second in total. Relative to a mission lasting tens of seconds, this confirms that the search can be issued as an occasional, non-blocking background task rather than something that interferes with the 5 ms control step.

Controller tuning study. The purpose of the host workflow is to recover a good cascaded-PID gain set even from a poor start. We therefore began from a deliberately careless gain set expected to crash:

$$K_{init} = \begin{bmatrix} 1.0 & 1.0 & 1.0 \\ 1.0 & 1.0 & 1.0 \\ 10.0 & 10.0 & 10.0 \\ 30.0 & 0.0 & 5.0 \\ 30.0 & 0.0 & 5.0 \\ 50.0 & 0.0 & 10.0 \end{bmatrix}, \tag{34}$$

with rows for the $x, y, z, \phi, \theta, \psi$ controllers and columns K_p, K_i, K_d. Running with variance $\sigma_K = 0$ (all 1024 rollouts identical to K_{init}) crashed every rollout, confirming the start is unusable. We then ran three stages, each centered on the previous best and with decreasing variance. Table II summarizes them: the combined cost (Eq. (28)) fell from 0.1782 to 0.0527 and the crash count from 886 to a single rollout out of 1024, each stage taking about 0.2 s.

TABLE II
HOST-DRIVEN GAIN-SEARCH STAGES ON $(0, 0, 0) \to (5, 5, 5) \to (5, 5, 2)$, $N = 1024$ ROLLOUTS PER STAGE.

Stage	Variance σ_K	Combined cost	Crashed	Time
Init (σ=0)	0.0	— (all crash)	1024 / 1024	∼0.2 s
Iteration 1	1.0	0.1782	886 / 1024	∼0.2 s
Iteration 2	0.5	0.0790	217 / 1024	∼0.2 s
Iteration 3	0.2	0.0527	1 / 1024	∼0.2 s

The intermediate second-stage gains (centered on itera-

tion 1, $\sigma_K = 0.5$) were

$$K_2 = \begin{bmatrix} 1.0082 & 0.0500 & 1.0365 \\ 3.3880 & 0.0500 & 0.5300 \\ 20.7803 & 6.4709 & 4.4313 \\ 18.9236 & 0.0 & 0.2607 \\ 1.5 & 0.0 & 0.25 \\ 2.5 & 0.0 & 9.7585 \end{bmatrix}, \tag{35}$$

and the final gain set after the third stage was

$$K_{final} = \begin{bmatrix} 2.7435 & 0.0328 & 0.6597 \\ 10.9371 & 0.0349 & 0.5381 \\ 24.9835 & 1.4990 & 2.2077 \\ 13.9575 & 0.0 & 0.1970 \\ 1.6820 & 0.0 & 0.2358 \\ 1.1693 & 0.0 & 9.8145 \end{bmatrix}. \tag{36}$$

Figures 10 and 11 show the best first-iteration controller: the 3D path reaches all waypoints but the per-axis response, especially altitude, still shows a transient. Figures 12 and 13 show the second iteration, and Figs. 14 and 15 the third, where tracking is tighter and overshoot is reduced, matching the lower cost and near-zero crash count in Table II.

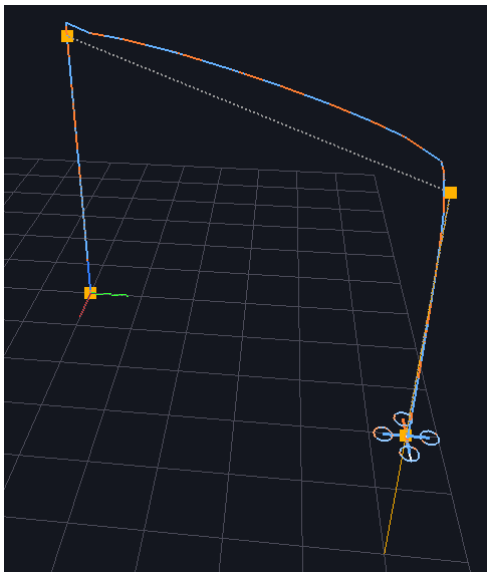

Fig. 10. 3D trajectory of the best first-iteration controller ($\sigma_K = 1$), rendered by the host viewer.

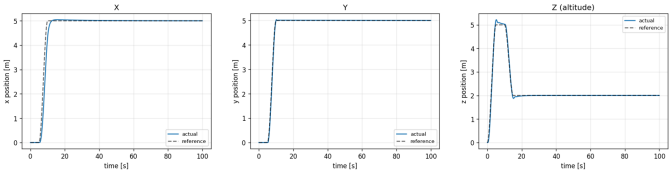

Fig. 11. Per-axis (x, y, z) tracking of the best first-iteration controller; the altitude channel shows the largest transient.

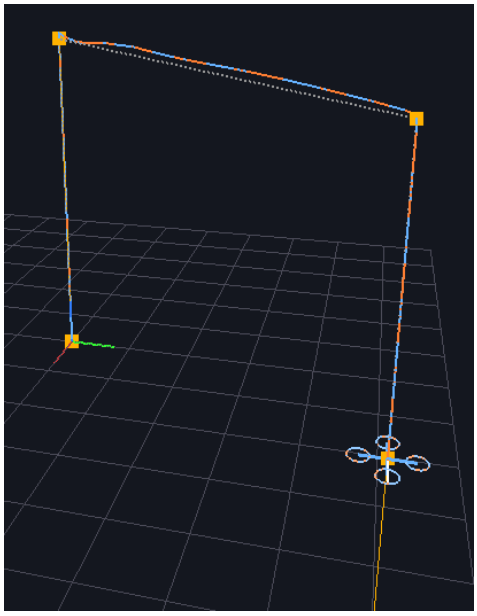

Fig. 12. 3D trajectory of the second-iteration controller ($\sigma_K = 0.5$).

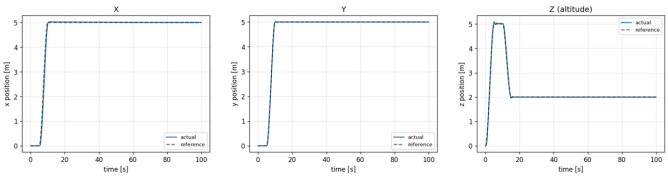

Fig. 13. Per-axis tracking of the second-iteration controller; fewer crashes and a lower cost than iteration 1.

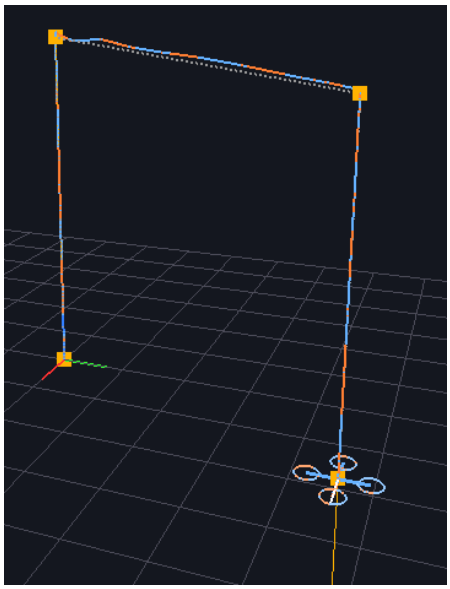

Fig. 14. 3D trajectory of the third-iteration controller ($\sigma_K = 0.2$); the path is visibly straighter than Fig. 10.

Per-thread footprint of the offloaded kernels (embedded constraint). The embedded requirement NFR-02 asks that each rollout thread fit within a 2 KB budget, so that a large N can run on a small device without spilling to global memory.

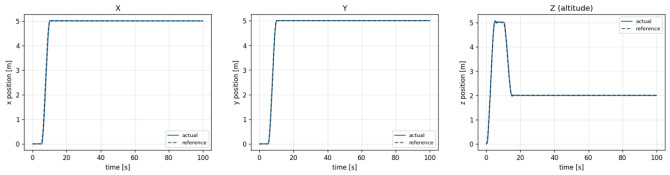

Fig. 15. Per-axis tracking of the third-iteration controller; overshoot and settling are improved over Fig. 11.

We measured the footprint at build time with `-Xptxas -v`. Table III reports it: the rollout kernel uses 63 registers and 412 B of constant memory (664 B total) and the scoring kernel 32 registers and 486 B of constant memory (604 B total) — both well under 2 KB. This is the key device-independent evidence that the offloaded workload is appropriate for a resource-constrained heterogeneous platform.

TABLE III
MEASURED PER-THREAD GPU RESOURCE USAGE (PTXAS, `-Xptxas -v`) OF THE OFFLOADED KERNELS AGAINST THE 2 KB BUDGET (NFR-02).

Kernel	Registers	Const. mem	Total	$<$ 2 KB?
Rollout / trajectory gen.	63	412 B	664 B	✓
Scoring	32	486 B	604 B	✓

B. Jetson Nano Performance Results

After concluding our simulation environment tests, we have managed to get a build on NVIDIA Jetson Nano board. We have removed the 3D plotting part and built the main algorithm only. Also, we have reduced the number of steps from 20000 to 5000, because the high number of samples resulted in timeouts making our application fail. This didn't cause any unfinished trajectory problems as the desired trajectory $(0,0,0) \rightarrow (5,5,5) \rightarrow (5,5,2)$ can fit into a 25 second episode, it just spends less time in steady state in comparison to our 20000 step 100 seconds episodes. The test flow remained the same with our Jetson Nano tests, again, we began from a deliberately careless gain set expected to crash:

$$K_{init} = \begin{bmatrix} 1.0 & 1.0 & 1.0 \\ 1.0 & 1.0 & 1.0 \\ 10.0 & 10.0 & 10.0 \\ 30.0 & 0.0 & 5.0 \\ 30.0 & 0.0 & 5.0 \\ 50.0 & 0.0 & 10.0 \end{bmatrix}, \tag{37}$$

After three iterations following the same steps, the resulting PID gain set from our Jetson Nano is shown in the following matrix.

$$K_{final} = \begin{bmatrix} 0.922 & 4.943 & 0.636 \\ 3.825 & 0.054 & 0.235 \\ 21.803 & 0.383 & 0.918 \\ 25.566 & 0.0 & 0.359 \\ 2.325 & 0.0 & 0.279 \\ 19.851 & 0.0 & 13.180 \end{bmatrix} \tag{38}$$

TABLE IV
JETSON NANO GAIN-SEARCH STAGES ON $(0,0,0) \rightarrow (5,5,5) \rightarrow (5,5,2)$, $N = 1024$ ROLLOUTS PER STAGE.

Stage	Variance σ_K	Combined cost	Crashed	Time
Iteration 1	1.0	0.306	875 / 1024	∼1.07 s
Iteration 2	0.5	0.127	144 / 1024	∼0.99 s
Iteration 3	0.2	0.083	0 / 1024	∼1.03 s

Then we have plotted the resulting trajectory from these gains in our desktop application and got 2D X-Y-Z trajectory plots and a 3D flight graph. The resulting trajectory is desirable, but it has some jitter around the desired trajectory. The combined cost in the end of desktop simulations was 0.0527 and the combined cost from Jetson Nano tests is 0.083 and as a result, we got a slightly worse performance on Jetson. However, since there are architectural and hardware differences between our desktop and Jetson Nano, we think that this difference is not that important.

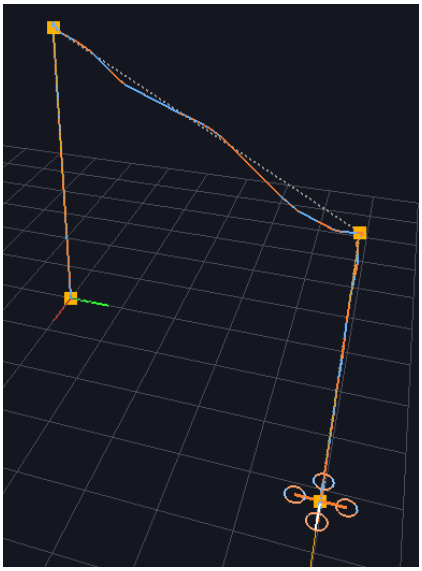

Fig. 16. 3D trajectory of the gain set result from Jetson Nano $(0,0,0) \rightarrow (5,5,5) \rightarrow (5,5,2)$.

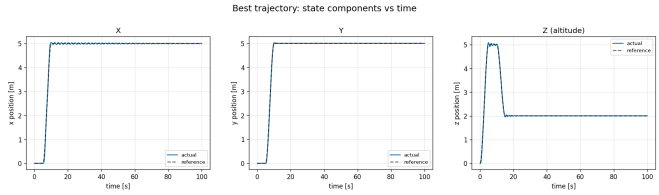

Fig. 17. Per-axis tracking of the gain set result from Jetson Nano

C. Comparative Analysis

Before vs. after tuning (own baseline). The clearest comparison is internal, on the same host and mission, before and after the search. The unmodified initial controller crashes

100% of rollouts; after three sub-second stages the host selects a controller that crashes 0.1% of rollouts and tracks all waypoints (Table II, Figs. 10–15). This before/after-optimization comparison shows the host turning an unusable controller into a usable one.

Configuration comparison (variance schedule). The three stages also compare configurations of our own method. A wide variance ($\sigma_K = 1$) explores broadly but wastes most rollouts ($886/1024$ crash); a narrow variance ($\sigma_K = 0.2$) refines locally with far fewer crashes ($1/1024$) and lower cost. This supports a multi-stage, decreasing-variance schedule over a single fixed variance.

Comparison with related work. To compare against the literature, the host re-ran the tuned controller ($\sigma_K = 0$) on the mission $(0,0,0) \rightarrow (2,2,2) \rightarrow (2,2,0)$, which matches trajectories used by Doukhi et al. [9] and Babu et al. [4]. Our tracking is shown in Figs. 18 and 19, and the reference results from Doukhi et al. are reproduced for comparison in Fig. 20. Qualitatively, our x, y, z step responses reach their targets with small overshoot and short settling, comparable to the self-tuning controllers in those works. The comparison is qualitative because the dynamics, disturbances, and reference timing differ, but it indicates that gains found on one mission generalize reasonably to another rather than overfitting a single reference.

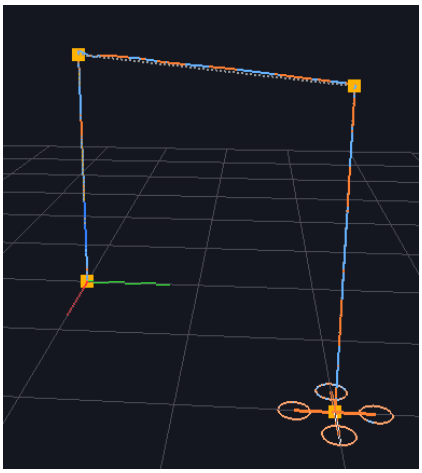

Fig. 18. 3D trajectory of the tuned controller on the comparison mission $(0,0,0) \rightarrow (2,2,2) \rightarrow (2,2,0)$.

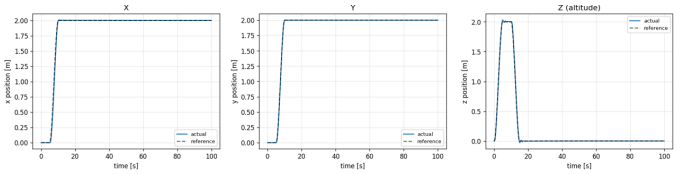

Fig. 19. Per-axis tracking of the tuned controller on the comparison mission.

Planned vs. obtained. Relative to the proposal, the embedded objectives that depend only on the host workflow and the offloaded kernels were met (a real-time-safe, non-blocking orchestration; a fast host-observed batch time; a working host-side selector; a build-verified compact footprint) on both desktop and on Jetson Nano board. Only thing missing from our results is the CPU based Monte Carlo timing results vs. our proposed CPU + GPU algorithm.

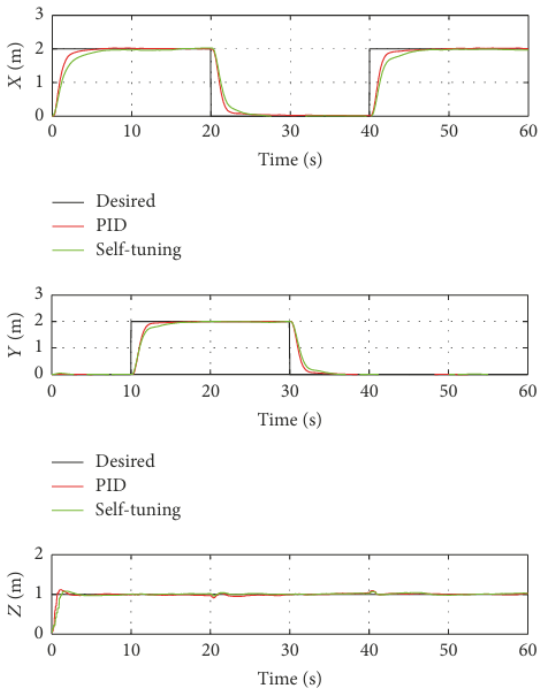

Fig. 20. Reference per-axis tracking reported by Doukhi et al. [9], shown for qualitative comparison.

D. Discussion

What worked. Our CPU + GPU gain optimization algorithm worked: the CPU ran the real-time control and simulation loop while the GPU ran an offloaded gain search that recovered a stable controller from an unstable start in under a second, within a Jetson-scale memory budget. Three decisions were decisive. First, the offload is advisory and issued between control periods, so the real-time loop is never blocked. Second, the host performs the final $\arg\min$ selection and only ever adopts a non-crashed candidate, because the scoring stage acts as a stability filter as well as an evaluator. Third, the per-rollout state was kept small enough that the offloaded kernels stay in registers/constant memory, which is what lets a large N run on a small device.

Embedded-systems concerns. The result connects directly to embedded concerns. *Real-time behavior:* decoupling the loop from the search and exchanging only a small gain vector protects the 5 ms deadline. *Resource constraints and memory:* since one rollout maps to one thread, memory grows linearly with N and the horizon T; the measured sub-1 KB footprint is what makes a large N feasible, and returning only compact metrics for non-winning candidates is the natural way to push N higher on a small device. *Reliability:* the advisory offload plus the crash filter give graceful degradation. *Numerical*

stability: forward Euler on the host and RK4 on the device both diverge under aggressive gains, so bounding the candidate range and penalizing crashes is essential.

Trade-offs and surprises. The main trade-off is exploration vs. stability in σ_K: a wide variance explores but wastes rollouts on crashes, a narrow one refines but cannot escape a poor basin. The high early crash count ($886/1024$) was expected from the deliberately bad start, but it also shows that raw random sampling is inefficient. A second trade-off is that the selected gains depend on the cost weights (α, β, γ): an RMSE-dominant weighting yields aggressive controllers, an overshoot/crash-dominant one yields conservative controllers, so the weights are effectively part of the design.

Limitations. Three stand out. (1) *Slower performance on Jetson Nano.* The run time for desktop simulations are around 0.2 seconds while the Jetson Nano tests has an average around 1.05 seconds. The performance can be even worse with longer trajectories so in order to compute 1024 trajectories under a second the trajectories must be kept short and this means the entire trajectory must be divided into serial parts and the optimizing algorithm must be used sequentially through these parts. (2) *Perturbations disabled.* To isolate gain selection we removed the environmental perturbations, so the selected gains are tuned for nominal conditions and are not robustness-optimal. (3) *Random local search.* Sampling only around the current center can stall in a local region if the start is very poor or σ_K is mis-set. These define the future work.

E. Reproducibility

The work is simulation-only on desktop and Jetson Nano, also it is deterministic for a fixed seed (NFR-04), so it can be reproduced as follows.

Toolchain and environment. The simulation environment consists of C++14 with CUDA C/C++, built in Visual Studio 2019 (toolset v142, CUDA v10.2) on Windows with a CUDA-capable NVIDIA GPU. Reused libraries: GATE (using Eigen v3.3.8 and providing the RK4 integrator), GLFW v3.4 for the 3D viewer, and Python 3.11 with Matplotlib 3.10.9 for the 2D plots. The same C++/CUDA sources build under the JetPack (Ubuntu) toolchain for a Jetson port.

Configuration and parameters. Inputs come from `config.txt` (Algorithm 1): the 18 base PID gains, the rollout count N, the sampling variance σ_K, and the seed; the four mission waypoints are entered at runtime. Fixed parameters: $m = 1.4\,\text{kg}$, $L = 1.0\,\text{m}$, $I_x = I_y = 0.0116$, $I_z = 0.0232\,\text{kg}\cdot\text{m}^2$, $k = 10^{-6}$, $g = 9.8\,\text{m/s}^2$; step size $h = \Delta t = 0.005\,\text{s}$; cost weights $(\alpha, \beta, \gamma) = (1,\, 0.05,\, 0.5)$. The reported runs use $N = 1024$ and the variance schedule $\{1.0, 0.5, 0.2\}$, with the multiplicative factor clamped to ≥ 0.05 to avoid non-positive gains.

Running the experiments. Build the CUDA project, set the gains/variance/seed in `config.txt`, run the executable, and enter the waypoints. The host samples candidates, launches the rollout and scoring kernels, performs the $\arg\min$ selection over non-crashed rollouts, writes `best_trajectory.csv`, and opens the GLFW viewer with the winning flight. The footprint numbers in Table III are reproduced by compiling with `-Xptxas -v` and reading the build log; the 2D plots are regenerated by running the Matplotlib script on `best_trajectory.csv`.

The following link https://github.com/taischken/Quadrotor-Control-and-Evaluation-Simulation-on-NVIDIA-Jetson/blob/main/lw_gate_jetson.zip can be visited to get the Jetson Nano build core for our project.

VIII. Conclusion and Future Work

This project implemented a heterogeneous CPU–GPU quadrotor simulation and PID gain-evaluation framework for an embedded platform of the NVIDIA Jetson class, studied here from the CPU/host side. The host runs the real-time nominal loop — reference generation, cascaded PID control, motor mixing, and forward-Euler dynamics at a 5 ms step — and acts as the coordinator that samples candidate gains, offloads their evaluation to the GPU, selects the best result, and updates the controller. The offloaded accelerator is a re-engineered GATE pipeline whose perturbation modules were removed so that each thread evaluates one candidate gain set and a scoring stage ranks candidates by position RMSE, settling time, overshoot, and crash status.

The main result is that this host-driven workflow recovers a usable controller from a deliberately unstable start in under a second, using a multi-stage decreasing-variance schedule, while the offloaded kernels stay within a sub-1 KB per-thread footprint (664 B and 604 B) against a 2 KB budget. From the embedded perspective the important findings are that the real-time loop is structurally shielded from the asynchronous search, the host orchestration is cheap enough ($\sim$0.2 s on desktop and $\sim$1.05 s on Jetson Nano per batch) to run as a background task, and the offloaded workload's resource cost is small and build-verified — the property that makes the design deployable on a small unified-memory device.

What was successfully completed: the host-side simulator, cascaded controller, reference generator, orchestration driver, best-gain selector, and CPU validation/visualization; the re-engineered GATE offload with its rollout and scoring kernels; the gain-search study; and the build-time footprint verification. These are implemented in Jetson Nano and desktop PC. What was not completed: even though we have built and run our algorithm on our target embedded device we have not tested it on a real environment with a quadrotor. Also, on-device energy (`tegrastats`), on-device control-loop and orchestration latency are not done. The GPU-vs-CPU crossover was not measured; and the environmental-perturbation robustness path, though preserved in the architecture, was disabled to isolate gain selection. The main limitations follow: results reflect nominal conditions on non-target hardware, and the search is a random local exploration whose quality depends on the initial gains, the variance schedule, the rollout count, and the cost weights.

Future work targets these openings. First, port to and benchmark on an actual quadrotor controlled by Jetson Nano.

Second, re-enable the perturbation modules to evaluate robustness under wind, sensor noise, and mass/inertia uncertainty. Third, replace pure random sampling with adaptive variance scheduling, multi-stage refinement, or evolutionary optimization for a more sample-efficient search. Finally, extend the host toward true online adaptation, in which the GPU periodically evaluates fresh candidates while the host continues the nominal loop without interruption.

ACKNOWLEDGMENTS

Through this project we gained hands-on experience in embedded real-time control-loop design, CPU–GPU workload partitioning on a heterogeneous platform, quadrotor dynamics modeling, and cascaded PID controller design. We developed a clearer understanding of how a host CPU can run a real-time control and simulation loop while orchestrating an offloaded parallel search as an advisory coprocessor, and of the practical limits of resource-constrained heterogeneous platforms in terms of timing, per-thread memory footprint, synchronization overhead, and numerical stability.

REFERENCES

[1] K. Masuda and K. Uchiyama, "Attitude control system for quadrotor using robust Monte Carlo model predictive control," *Actuators*, vol. 13, no. 11, p. 443, 2024.

[2] R. Melzer, M. Gandhi, R. Schlossman, K. Williams, and J. Parish, "CUDA for rapid controller robustness evaluation: A tutorial," *Sandia National Laboratories Technical Report*, 2021.

[3] G. Delgado-Reyes, J. S. Valdez-Martínez, M. Hernández-Pérez, K. R. Pérez-Daniel, and P. J. García-Ramírez, "Quadrotor real-time simulation: A temporary computational complexity-based approach," *Mathematics*, vol. 10, no. 12, p. 2032, 2022.

[4] V. M. Babu, K. Das, and S. Kumar, "Designing of self tuning PID controller for AR drone quadrotor," *2017 18th International Conference on Advanced Robotics*, pp. 167–172, 2017.

[5] M. Rizzello and M. Molaei, "Motor-quadrotor simulations based on FOC and cuckoo PID tuning," *IEEE International Conference*, 2024.

[6] A. Iyer, "Modelling, simulation, and implementation of PID controller on quadrotors," *ResearchGate Preprint*, 2021.

[7] A. T. Reenberg, J. K. Huusom, and J. B. Jorgensen, "A high-performance Monte Carlo simulation toolbox for uncertainty quantification of closed-loop systems," *2022 IEEE Conference on Decision and Control*, 2022.

[8] Unknown, "Dynamic response optimization for quadrotor UAVs using a parameter self-tuning fuzzy PID control method," *IEEE*, 2024.

[9] O. Doukhi and D. J. Lee, "Intelligent controller design for quad-rotor stabilization in presence of parameter variations," *Journal of Advanced Transportation*, vol. 2017, p. 4683912, 2017.

[10] G. M. Chari, A. G. Kamath, P. Elango, and B. Acikmese, "Fast Monte Carlo analysis for 6-DoF powered-descent guidance via GPU-accelerated sequential convex programming," *AIAA SciTech Forum*, 2024.

[11] NVIDIA, "Getting started with Jetson Nano 2GB developer kit," *NVIDIA Developer Documentation*, 2024.